

History Lesson

Python came on to the scene in the late 80s and implementation began sometime in the last year of the decade

No foo bar

Instead common metasyntactic variables in Python are spam, ham, and eggs

```
>>> 3.1 in a_list
True
>>> 2.7 in a_list
False
>>> a_list.index( 'molurus' )
1
```

Tuples in Python

Python also includes a very useful way of handling data called a tuple. A tuple is simply a comma separated sequence of data. A variable can be made a tuple as simply as:

```
a = 1, 2, 3, "four"
```

Quite similarly, you can get back these values in individual variables as:

```
x, y, z, w = a
```

Using tuple Python can accomplish stuff that is usually not found in other languages, such as the aforementioned multiple variable assignments in a single line, or returning multiple values from a function using the return statement.

Flow control:

In Python flow control is accomplished using the if, for and while statements. How they are used will become clear over the following examples:

The if statement:

```
if inp == "yes":
    print("You entered 'Yes' as
your response.")
else inp == "no":
    print("You entered 'No' as
your response.")
else:
    print("Invalid response.")
```

The for statement:

The 'for' statement in Python differs from that of many other languages. Where you'd have something like:

```
for(i=0;i<11;i+=3)
In Python you'll have:
for i in range(0,11,3)
```

The range function can be used as range(start, end, step) to get the same outcome.

In Python the for statement can very easily be used to iterate over lists as follows:

```
for item in a_list:
    print(item)
```

The while function:

The while function has already been used by us before. It has a simple syntax:

```
while condition_is_true:
    do()
    stuff()
```

Defining functions in Python:

Python has a simple, consistent and easy to understand syntax for defining functions. The following is our previously used Fibonacci sequence code as a function:

```
def fibo(n):
    a, b = 0, 1
    while b < n:
        a, b = b, a+b
    return a
```

Unlike many other languages, functions in Python can not only receive multiple arguments, but they can return multiple results as well! So if in our example above, we – for some strange reason – wanted to return both a and b, we could do that by simply returning them as:

```
return a, b
```

This utilizes a concept of Python called tuples, which we have mentioned before. The result could be retrieved as:

```
c = fibo(10)
d, e = fibo(100)
```

Where c will then be assigned the value of the tuple (8,13), and c and d will respectively be assigned values of 89 and 144 respectively. An effective example would be a function returning coordinates (x, y) or (x, y, z).

Moving forward with Python

What we have covered here is just the bare minimum of Python, you have a great journey ahead, exploring this marvellous language and its libraries. However to give you a taste of what you can do in just a few minutes, here we will write a simple Python program to input a CSV file and output it as an HTML table:

```
import csv
def csv_to_html(csv_data):
    table = "<table>"
    for row in csv_data:
        table += "<tr>"
        for column in row:
            table +=
```

```
"<td>" + column + "</td>"
        table += "</tr>"
    table += "</table>"
    return table
reader = csv.reader(open("C:/
data.csv"))
html_output = "<html><head>"
html_output += "<title>csv
table</title>"
html_output += "</head><body>"
html_output += csv_
to_html(reader)
html_output += "</body></html>"
print(html_output)
```

While this uses a lot of Python features which we haven't studied, you will notice that the code is still easy to understand. We have used import to import CSV functionality of the Python standard library. We have used the open function which can be used to open files. Our csv_to_html function reads each item of the CSV file and constructs a table from the data, which it returns as a string. Around that we will pad the standard minimum HTML code and print it.

Right now in this simple example we have hardcoded the file to open, and we are printing the output to screen instead of saving it to a file. With just a little more code, you can extend this program to open any file and output to any file by providing command-line parameters.

Currently you can test this file by running it as follows:

```
python csv2html.
py > htmlfile.html
```

Then open htmlfile.html; of course you will need to have a CSV file at 'C:\data.csv' and it will need to be formatted in a standards-compliant way (saving from Excel might not work, use Google Docs).

Conclusion

Python is a great language to begin with, and requires little effort to get started. The wide abundance of libraries available for Python, from networking and GUI, to mathematics and data processing, make it simply to make application with a lot of functionality.

Learning Python won't go to waste, as it will always find a place in your workflow no matter which language you move on to. 